

Discrete-Math-Assignment-1

Student Information

- **學生姓名:** 彭靖凱
 - **學號:** E24134778
 - **授課教授:** 解巽評 教授
-

Question Set

離散數學程式作業 Q1: 由 CSV 檔讀取命題公式並產生真值表

本作業對應投影片中的 **Q1. Statements to truth table**, 要求如下:

- 從 `input1.csv` 讀取多個命題邏輯敘述 (欄位: `Number, statements`) 。
 - 對每一個敘述產生完整的真值表, 只輸出「最終答案」(T / F), 不需顯示推導過程。
 - 每一題的真值表在輸出檔中須以**空一行**分隔 (wrap after every question) 。
 - 輸出結果寫入 `output1.csv`。
-

一、必要套件與安裝說明

(Required packages and installation instructions)

1.1 執行環境

- **程式語言:** Python 3.11.5
- **執行環境:** Anaconda Python (Windows PowerShell 下測試執行)
- **作業系統:** Windows 10/11 (開發與測試皆於 Windows 上進行)

1.2 必要套件 (Required packages)

本作業程式僅使用 **Python 標準函式庫**:

- `csv`: 用來讀寫 CSV 檔 (`input1.csv` / `output1.csv`)
- `itertools`: 用來產生所有命題變數的 True/False 組合

因為只用到標準函式庫, **不需要額外安裝第三方套件** (不需要 `pip install` 任何東西) 。

1.3 安裝與環境設定步驟

(1). 安裝 Python 3.11 (或相容版本)

- 建議安裝 Anaconda (內建多數科學運算套件, 方便之後其他課程使用) 。
- 安裝完後, 可在終端機或 PowerShell 確認版本:

```
python --version
```

(2). 取得作業程式碼

- 方式一：從 GitHub Classroom 自動產生的 repo `clone` 到本機：

```
git clone <這次作業的 repo URL>
cd <repo 資料夾>
```

- 方式二：直接下載壓縮檔並解壓縮，進入解壓出的資料夾。

(3). 確認專案目錄內的重要檔案

在專案根目錄應至少包含：

- `Q1.py` — 主程式
- `input1.csv` — 作業要求的輸入檔（命題邏輯敘述）
- `output1.csv` — Q1 真值表輸出結果（執行後產生）
- `README.md` — Q1說明文件

二、程式使用說明

(Usage instruction for your code)

2.1 檔案結構

本作業目前僅實作 **Q1 真值表產生器**，相關檔案結構如下：

```
.
├─ Q1.py           # 主程式：讀 input1.csv, 產生 truth table, 寫 output1.csv
├─ input1.csv      # 輸入檔：儲存題號與命題敘述 (Number, statements)
├─ output1.csv     # 輸出檔：程式執行後產生的真值表
└─ README.md      # Q1說明文件
```

2.2 輸入檔 `input1.csv` 格式說明

`input1.csv` 第一行必須是標題列：

```
Number,statements
```

後面每一行代表一個要產生真值表的命題邏輯敘述，例如：

```
Number,statements
1, [(P→Q)∧(Q→R)]→(P→R)
2, [(P∨Q)∧¬P]→Q
3, [(P→Q)∧¬Q]→¬P
4, [(P∧Q)∨R]↔[(P∨R)∧(Q∨R)]
```

```
5, [P↔(Q∧R)] ↔ [(P→Q) ∧ (P→R) ∧ (Q→P) ∧ (R→P)]
6, [(P∧Q) ↔ (RVS)]
```

- **Number** 欄位: 題號 (例如 1, 2, 3, ...)
- **statements** 欄位: 命題邏輯公式字串

2.2.1 支援的符號

程式會先將輸入字串做「字元轉換與標準化」, 支援以下符號:

- **命題變數:**
 - 單一英數字元, 例如: P, Q, R, S, T, A, B, ...
 - 程式以 `token.isalnum()` 判斷是否為變數。
- **否定 (NOT):**
 - $\sim$ 或 $\neg$
- **且 (AND):**
 - $\wedge$
 - ^ 會被程式自動轉換為 $\wedge$ 處理
- **或 (OR):**
 - $\vee$
 - v 會被視為 $\vee$ 的同義符號
- **蘊涵 (IMPLIES):**
 - $\rightarrow$
 - > 會被程式轉換為內部的 $\rightarrow$
- **雙條件 (BICONDITIONAL):**
 - $\leftrightarrow$
 - <-> 會被程式轉換為內部的 $\leftrightarrow$
- **括號:**
 - $()$
 - $[]$ 會被程式視為 $()$ (中括號自動轉成小括號)

2.2.2 運算子優先順序

在轉成 postfix (後序表示) 時, 運算子優先順序設定如下:

- (1). 否定: $\sim$ 、 $\neg$
- (2). 且: $\wedge$ (以及被轉換的 ^)
- (3). 或: $\vee$ 、 v

- (4). 蘊涵： $\rightarrow$ (以及被轉換的 $\Rightarrow$)
 (5). 雙條件： $\leftrightarrow$ (以及被轉換的 $\Leftrightarrow$)

也就是：

NOT > AND > OR > IMPLIES > BICONDITIONAL

2.3 執行方式

請先切換到包含 `Q1.py` 與 `input1.csv` 的資料夾，再執行：

```
python Q1.py
# 或 (依系統設定)
python3 Q1.py
```

程式會：

- (1). 以 UTF-8 編碼讀取 `input1.csv`，檢查是否包含 `Number` 與 `statements` 兩欄。
- (2). 對每一列的命題公式：
 - 解析字串 $\rightarrow$ 轉成 token 串列
 - 使用 shunting-yard 演算法轉成 postfix 表示
 - 找出出現的所有命題變數 (例如 P, Q, R, S)
 - 使用 `itertools.product` 列舉所有 T/F 組合
 - 在每個組合下計算整個公式的真假值
- (3). 將結果寫入 `output1.csv`，並在終端機印出每個題號與每列的 T/F 結果。

2.4 輸出檔 `output1.csv` 格式說明

`output1.csv` 會被 **整個重寫 (overwrite)**，內容格式如下：

- 每一題的區塊包含：(1) 題頭列：#題號, 原始命題公式 (2) 欄位標題列：所有變數 (依字母排序) + `Result` (3) 若干資料列：每行為一組變數指派與該指派下的結果 (T 或 F) (4) 題與題之間會插入一個空白行 (僅有逗號)，方便閱讀

範例 (節錄)：

```
#1, [(P→Q)^(Q→R)]→(P→R)
P,Q,R,Result
F,F,F,T
F,F,T,T
F,T,F,T
F,T,T,T
T,F,F,T
T,F,T,T
T,T,F,T
T,T,T,T

#2, [(P∨Q)^(¬P)]→Q
P,Q,Result
```

F,F,T
F,T,T
T,F,T
T,T,T

#3, $[(P \rightarrow Q) \wedge \neg Q] \rightarrow \neg P$
P,Q,Result
F,F,T
F,T,T
T,F,T
T,T,T

#4, $[(P \wedge Q) \vee R] \leftrightarrow [(P \vee R) \wedge (Q \vee R)]$
P,Q,R,Result
F,F,F,T
F,F,T,T
F,T,F,T
F,T,T,T
T,F,F,T
T,F,T,T
T,T,F,T
T,T,T,T

#5, $[P \leftrightarrow (Q \wedge R)] \leftrightarrow [(P \rightarrow Q) \wedge (P \rightarrow R) \wedge (Q \rightarrow P) \wedge (R \rightarrow P)]$
P,Q,R,Result
F,F,F,T
F,F,T,F
F,T,F,F
F,T,T,T
T,F,F,T
T,F,T,T
T,T,F,T
T,T,T,T

#6, $[(P \wedge Q) \leftrightarrow (R \vee S)]$
P,Q,R,S,Result
F,F,F,F,T
F,F,F,T,F
F,F,T,F,F
F,F,T,T,F
F,T,F,F,T
F,T,F,T,F
F,T,T,F,F
F,T,T,T,F
T,F,F,F,T
T,F,F,T,F
T,F,T,F,F
T,F,T,T,F
T,T,F,F,F
T,T,F,T,T
T,T,T,F,T
T,T,T,T,T

#7, $[(P \wedge Q) \rightarrow (R \vee S \vee T)]$

```

P,Q,R,S,T,Result
F,F,F,F,F,T
F,F,F,F,T,T
F,F,F,T,F,T
F,F,F,T,T,T
F,F,T,F,F,T
F,F,T,F,T,T
F,F,T,T,F,T
F,F,T,T,T,T
F,T,F,F,F,T
F,T,F,F,T,T
F,T,F,T,F,T
F,T,F,T,T,T
F,T,T,F,F,T
F,T,T,F,T,T
F,T,T,T,F,T
F,T,T,T,T,T
T,F,F,F,F,T
T,F,F,F,T,T
T,F,F,T,F,T
T,F,F,T,T,T
T,F,T,F,F,T
T,F,T,F,T,T
T,F,T,T,F,T
T,F,T,T,T,T
T,T,F,F,F,F
T,T,F,F,T,T
T,T,F,T,F,T
T,T,F,T,T,T
T,T,T,F,F,T
T,T,T,F,T,T
T,T,T,T,F,T
T,T,T,T,T,T

```

- 每行的 **T** / **F** 與終端機中印出的 list（例如 `['F', 'T', 'T']`）相對應。
- **Result** 欄表示在該組變數值下整個命題公式的真假。

三、其他編譯語言之編譯方式

(Compile method if you use any other compiled language)

本次作業 **僅使用 Python 實作 Q1**，屬於直譯式語言，**不需要額外編譯步驟**。

- 執行方式僅需：

```
python Q1.py
```

四、程式邏輯簡介

1. `get_precedence(op)`

- **用途：**
回傳邏輯運算子 `op` 的優先順序，供中序轉後序（postfix）時使用。
 - **參數：**
 - `op`：字串，邏輯運算子（如 `~`、`^`、`v`、`->`、`<->` 等）。
 - **回傳值：**
 - `int`：該運算子的優先權數值（數字越大優先權越高），若不是已知運算子則回傳 `-1`。
-

2. `tokenize(expression: str)`

- **用途：**
將一個邏輯敘述字串切成 token 串列（變數、運算子、括號），並在這裡做符號的統一與標準化。
 - **處理內容：**
 - 將 `→` 轉成 `->`，`↔` 轉成 `<->`。
 - 將 `[`、`]` 當作 `(`、`)`。
 - 將 `^` 轉成 `^`。
 - 支援多字元運算子 `<->`、`->`。
 - 以 `isalnum()` 判斷變數（例如 `P`、`Q`、`R`）。
 - **參數：**
 - `expression`：字串，原始的命題邏輯公式。
 - **回傳值：**
 - `list[str]`：token 串列，例如 `['(', 'P', '->', 'Q', ')', '^', 'R']`。
-

3. `infix_to_postfix(tokens)`

- **用途：**
使用 Shunting-yard 演算法，將中序（infix）表示的運算式轉成後序（postfix）。
 - **參數：**
 - `tokens`：`list[str]`，由 `tokenize()` 產生的 token 串列。
 - **回傳值：**
 - `list[str]`：postfix 形式的 token 串列，方便後續以堆疊計算。
-

4. `evaluate_postfix(postfix, values)`

- **用途：**
在給定變數真值指派的情況下，計算一個 postfix 表達式的真假值。
- **邏輯：**
 - 遇到變數 token 時，從 `values` 字典取出對應的 `True/False` 推入堆疊。
 - 遇到否定運算子（`~`、`¬`）時，彈出一個值取反。
 - 遇到二元運算子（`^`、`v`、`V`、`->`、`<->`）時，彈出兩個值依規則計算後再推回堆疊。
- **參數：**
 - `postfix`：`list[str]`，postfix 表示的運算式。
 - `values`：`dict[str, bool]`，變數對應到布林值的映射，例如 `{'P': True, 'Q': False}`。

- **回傳值：**
 - `bool`：整個公式在該組變數指派下的結果 `True` 或 `False`。
-

5. `generate_truth_table(expression: str)`

- **用途：**

對單一命題公式產生完整真值表。
 - **處理流程：** (1). 呼叫 `tokenize()` 取得 token。 (2). 呼叫 `infix_to_postfix()` 轉成 postfix。 (3). 從 token 中找出所有出現的變數並排序。 (4). 利用 `itertools.product([False, True], repeat=len(variables))` 產生所有 T/F 組合。 (5). 對每一組組合呼叫 `evaluate_postfix()` 得到結果。
 - **參數：**
 - `expression`：字串，命題公式。
 - **回傳值：**
 - `(variables, rows)`:
 - `variables`: `list[str]`，例如 `['P', 'Q', 'R']`。
 - `rows`: `list[ (tuple[bool, ...], bool) ]`，每一列為 (變數真值 tuple, 結果 bool)。
-

6. `process_csv_to_output(input_filename="input1.csv", output_filename="output1.csv")`

- **用途：**

依作業規定，從輸入 CSV 讀取多題敘述，為每一題產生真值表，並寫入輸出 CSV。
 - **處理流程：** (1). 使用 `csv.DictReader` 讀取 `input_filename`，檢查是否有 `Number` 與 `statements` 欄位。 (2). 對每一列（每一題）取出題號與公式，存入 `questions` 列表。 (3). 開啟 `output_filename` 以 `csv.writer` 寫入：
 - 題頭列： `[#題號, 敘述]`
 - 標題列：變數名稱 + `Result`
 - 每一列：將 `True/False` 轉成 `T/F` 後寫出。
 - 題與題之間寫入一行空白列作為分隔。 (4). 同時在終端機 `print` 每一題的題頭與每列 `['T', 'F', ...]` 結果，最後印出檔案寫入完成訊息。
 - **參數：**
 - `input_filename`：輸入 CSV 檔名（預設 `"input1.csv"`）。
 - `output_filename`：輸出 CSV 檔名（預設 `"output1.csv"`）。
 - **回傳值：**
 - `None`（以副作用的方式產生檔案與終端機輸出）。
-

7. `if __name__ == "__main__":`

- **用途：**

讓程式在「直接執行 `Q1.py`」時，自動呼叫 `process_csv_to_output()`；若此檔案被當作模組匯入，則不會自動執行主流程。
- **行為：**

```
if __name__ == "__main__":  
    process_csv_to_output()
```

因此在一般情況下，只要在專案根目錄執行：

```
```bash  
python Q1.py
```

即可完成整個作業要求的流程。

---

# Discrete-Math-Assignment – Q2

---

## Student Information

- **學生姓名:** 彭靖凱
  - **學號:** E24134778
  - **授課教授:** 解巽評 教授
- 

## Question Set

離散數學程式作業 Q2: 判斷圖是否存在歐拉路徑 / 歐拉迴路並輸出路徑

程式檔: `Q2.py`

- 從 `input2.csv` 讀入「邊列表」(無向圖)
  - 依題號分組, 對每一張圖判斷是否存在
    - **Euler circuit (歐拉迴路)**
    - **Euler path (歐拉路徑)**
  - 若存在則利用 Hierholzer 演算法求出實際路徑
  - 將結果輸出到 `output2.csv`, 同時在終端機印出結論與路徑
- 

## 一、必要套件與安裝說明 (Q2)

(Required packages and installation instructions)

### 1.1 執行環境

- **程式語言:** Python 3.11 (3.8 以上皆可)
- **建議環境:** Anaconda Python (於 Windows PowerShell 下測試)
- **作業系統:** Windows 10 / 11

### 1.2 必要套件 (Required packages)

`Q2.py` 只使用 **Python 標準函式庫**:

- `csv`: 讀寫 `input2.csv`、`output2.csv`
- `collections.defaultdict`、`collections.deque`:
  - `defaultdict(list)`: 儲存圖的鄰接表
  - `deque`: 在連通性檢查中進行 BFS

✅ 以上皆為內建模組, **不需要安裝任何第三方套件**, 不用執行 `pip install`。

### 1.3 安裝與環境設定步驟

#### 1. 安裝 Python 3.11 (或相容版本)

建議安裝 Anaconda, 安裝完成後於終端機 / PowerShell 確認版本:

```
python --version
```

## 2. 取得作業程式碼與資料檔

```
git clone <repo-url>
cd <repo-folder>
```

或下載壓縮檔後解壓縮並進入資料夾。

## 3. 確認 Q2 相關檔案位於同一資料夾

- `Q2.py` — Q2 主程式
- `input2.csv` — Q2 輸入的邊列表（多個題目）
- `output2.csv` — Q2 的結果輸出檔（執行後產生）
- `README.md` — Q2說明文件

---

# 二、程式使用說明（Q2）

(Usage instruction for your code)

## 2.1 執行方式

在包含 `Q2.py` 與 `input2.csv` 的資料夾中執行：

```
python Q2.py
或依系統設定：
python3 Q2.py
```

Windows PowerShell 範例：

```
python .\Q2.py
```

程式會：

1. 從 `input2.csv` 依題號讀入所有邊。
2. 對每一題的圖判斷是否存在 Euler path / circuit。
3. 若存在，使用 Hierholzer 演算法找出一條實際路徑。
4. 將結果寫入 `output2.csv`，並在終端機印出類似：

題目： 1

結論： 存在歐拉迴路

歐拉路徑/迴路： A -> C -> B -> A

-----

執行結束後，會在同一路徑看到 **output2.csv**。

---

## 2.2 輸入檔 **input2.csv** 格式說明

**input2.csv** 使用「**題號 + 邊列表**」的文字格式，題與題之間用空行分隔：

#1

A,B

B,C

C,A

#2

A,B

B,C

C,D

D,A

A,C

#3

A,B

B,C

C,D

D,E

E,A

A,C

C,E

E,B

D,B

A,D

#4

A,B

B,C

C,D

D,E

#5

A,B

B,C

C,D

D,A

A,E

E,B

B,D

```

D,A
C,E
E,A
A,B

#6
A,B
B,C
C,D
D,E
E,F
F,A
A,C
C,E
E,A
F,B

```

- 每一題以 **#題號** 開頭，例如 **#1, #2, #3**。
- 接著若干行，每一行代表一條**無向邊** **u,v**:
  - 逗號分隔兩個頂點名稱（通常為單一大寫字母 **A~Z**）。
  - **A,B** 代表邊 {A,B}，與 **B,A** 等價。
- 題目結束與下一題之間可以有一到多行空白行。
- 檔案中出現的 ... 為示意（實際作業檔不會出現 ... 此行）。

程式在 `read_q2_input_text()` 中會：

1. 遇到 **#** 開頭的行，辨識出當前題號。
2. 遇到含 **,** 的行，解析為一條邊 (**u, v**) 並加入對應题目的邊列表。
3. 空白行只作為分隔，不影響邏輯。

## 2.3 輸出檔 `output2.csv` 格式說明

`output2.csv` 的欄位為：

```
question,type,path
```

每一列代表一題的判斷結果，例如：

```

題目： 1
結論： 存在歐拉迴路
歐拉路徑/迴路： A -> C -> B -> A

題目： 2
結論： 存在歐拉路徑
歐拉路徑/迴路： A -> C -> D -> A -> B -> C

題目： 3
結論： 存在歐拉迴路

```

歐拉路徑/迴路：A -> D -> B -> E -> C -> A -> E -> D -> C -> B -> A

題目：4

結論：存在歐拉路徑

歐拉路徑/迴路：A -> B -> C -> D -> E

題目：5

結論：存在歐拉路徑

歐拉路徑/迴路：B -> A -> B -> D -> A -> E -> A -> D -> C -> E -> B -> C

題目：6

結論：存在歐拉路徑

歐拉路徑/迴路：B -> F -> A -> E -> C -> A -> B -> C -> D -> E -> F

- **question**：題號（從 #1, #2, ... 解析而來）。
- **type**：
  - "歐拉迴路"：存在 Euler circuit（歐拉迴路）
  - "歐拉路徑"：存在 Euler path 但無迴路
  - "none"：不存在 Euler path 或 circuit
- **path**：
  - 若 **type** 為 歐拉迴路 或 歐拉路徑，會輸出路徑中依序走訪的頂點，例如：  
A -> C -> D -> A -> B -> C
  - 若 **type** == "none"，則通常輸出 N/A 或空字串（依實作而定）。

### 三、其他編譯語言之編譯方式（Q2）

(Compile method if you use any other compiled language)

本題使用 **Python**（直譯式語言）實作，**沒有**使用 C / C++ / Java 等編譯型語言，因此 **不需要額外編譯步驟**。

- 只需在含有 **Q2.py** 與 **input2.csv** 的資料夾中執行：

```
python Q2.py
```

若未來在同一 repo 中加入以 C++ 或其他語言撰寫的版本，建議在本節額外標明：

- 編譯指令（例如 `g++ Q2.cpp -std=c++17 -O2 -o Q2`）
- 執行指令（例如 `./Q2`），以及讀寫的檔名與格式。

### 四、類別與函式說明（Q2）

(Functions description)

本程式未定義自訂類別，主要函式如下：

#### 4.1 `build_graph(edges)`

- **功能：**  
將邊列表 `edges` 轉成無向圖的鄰接表 (adjacency list) 。
  - **輸入：**
    - `edges: list[tuple[str, str]]`, 每一項為一條無向邊 ( $u, v$ )。
  - **輸出：**
    - `adj: defaultdict(list)`, `adj[u]` 是與頂點  $u$  相鄰的所有頂點。
- 

## 4.2 `is_connected(adj)`

- **功能：**  
檢查在只考慮「度數  $> 0$  的頂點」時，圖是否連通。
  - **步驟概念：**
    1. 找任一有邊的頂點作為起點。
    2. 使用 `deque` 進行 BFS，將能到達的頂點加入 `visited`。
    3. 若所有度數  $> 0$  的頂點都在 `visited` 中，則視為連通。
- 

## 4.3 `euler_type(adj)`

- **功能：**  
判斷圖是否存在 Euler circuit / path / none，並給出適合的起點。
  - **核心邏輯：**
    1. 先呼叫 `is_connected(adj)`，若不連通則回傳 `"none"`。
    2. 數每個頂點的度數：
      - 若所有頂點度數皆為偶數  $\rightarrow$  `type = "circuit"`，從任一有邊的頂點開始。
      - 若恰好有兩個奇數度頂點  $\rightarrow$  `type = "path"`，從其中一個奇數頂點開始。
      - 其他情況  $\rightarrow$  `type = "none"`。
- 

## 4.4 `find_eulerian_trail_or_circuit(adj, start)`

- **功能：**  
已知圖存在 Euler path 或 circuit，且給定起點 `start`，  
使用 **Hierholzer 演算法** 求出一條實際的 Euler 路徑 / 迴路。
  - **輸入：**
    - `adj`：圖的鄰接表（會先複製一份以避免破壞原資料）。
    - `start`：起點頂點（由 `euler_type()` 提供）。
  - **輸出：**
    - `path: list[str]`，依序走訪的頂點，例如：`["A", "C", "B", "A"]`。
- 

## 4.5 `read_q2_input_text(input_filename="input2.csv")`

- **功能：**  
讀取 `input2.csv`，依 #題號 將邊列表分組。
- **輸入：**
  - `input_filename`：輸入檔名（預設 `"input2.csv"`）。
- **輸出：**

- `edges_by_q: dict[str, list[tuple[str, str]]]`,  
例如 `{"1": [("A", "B"), ("B", "C"), ("C", "A")], "2": [...], ...}`。

---

#### 4.6 `process_q2(input_filename="input2.csv", output_filename="output2.csv")`

- **功能：**

Q2 的主流程：讀取輸入、判斷每一題的 Euler 型態、求路徑並輸出。

- **步驟概念：**

1. 呼叫 `read_q2_input_text()` 取得 `edges_by_q`。
2. 依題號排序後逐題處理：
  - 使用 `build_graph()` 建立圖。
  - 使用 `euler_type()` 判斷 "circuit" / "path" / "none" 及起點。
  - 若不是 "none", 呼叫 `find_eulerian_trail_or_circuit()` 求得路徑。
  - 將結果整理成一行 `[question, type, path_str]` 加入 `out_rows`。
  - 同時在終端機印出「題目、結論、路徑」。
3. 將 `out_rows` 寫入 `output_filename` (CSV 格式) 。

---

#### 4.7 `if __name__ == "__main__":`

- **功能：**

讓檔案在「直接執行」時自動跑 Q2 主流程：

```
if __name__ == "__main__":
 process_q2()
```

因此使用時只需：

```
python Q2.py
```

便會完成所有題目的判斷與輸出。

# Discrete-Math-Assignment – Q3

---

## Student Information

- **學生姓名:** 彭靖凱
  - **學號:** E24134778
  - **授課教授:** 解巽評 教授
- 

## Question Set

離散數學程式作業 Q3：由 preorder / inorder 重建二元樹、輸出 postorder 並畫出樹形圖

本作業對應投影片中的題目：

給定每一題的 **前序走訪 (preorder)** 與 **中序走訪 (inorder)** 序列：

- 重建對應的二元樹
- 計算並輸出 **後序走訪 (postorder)** 序列
- 產生對應的二元樹圖片 `tree_*.png`

主程式檔： `Q3.py`

---

## 一、必要套件與安裝說明 (Q3)

(Required packages and installation instructions)

### 1.1 執行環境

- **程式語言:** Python 3.11 (3.8 以上皆可)
- **建議環境:** Anaconda Python (於 Windows PowerShell 下測試)
- **作業系統:** Windows 10 / 11

### 1.2 必要套件 (Required packages)

`Q3.py` 使用下列 Python 模組：

- 標準函式庫：
  - `csv`：讀寫 `input3.csv` / `output3.csv`
  - `typing`：型別註解 `List`, `Optional`, `Dict`, `Tuple`
- 第三方套件：
  - `networkx`：建立樹的節點與邊，協助畫圖
  - `matplotlib` (`matplotlib.pyplot`)：實際繪製與輸出樹形圖

`csv` 與 `typing` 為 Python 內建模組；

**必須額外安裝 `networkx` 與 `matplotlib` 才能畫圖。**

### 1.3 安裝與環境設定步驟

## 1. 安裝 Python 3.11（或相容版本）

建議使用 Anaconda，安裝後先確認版本：

```
python --version
```

## 2. 安裝第三方套件

使用 `pip` 安裝（在終端機 / PowerShell 中）：

```
pip install networkx matplotlib
```

## 3. 取得作業程式碼與資料檔

```
git clone <repo-url>
cd <repo-folder>
```

或下載壓縮檔解壓縮後進入資料夾，確認以下檔案存在：

- `Q3.py` — Q3 主程式
- `input3.csv` — 各題 preorder / inorder 序列
- （執行後產生）`output3.csv` — 各題 postorder 序列
- （執行後產生）`tree_1.png` ~ `tree_5.png` — 每題的二元樹圖片

---

## 二、程式使用說明（Q3）

(Usage instruction for your code)

### 2.1 執行方式

在含有 `Q3.py` 與 `input3.csv` 的資料夾中執行：

```
python Q3.py
或依系統設定：
python3 Q3.py
```

Windows PowerShell 範例：

```
python .\Q3.py
```

程式會：

1. 從 `input3.csv` 讀入每一題的 preorder 與 inorder 序列。
2. 重建各題的二元樹。
3. 計算後序走訪 (postorder) 並儲存在 `output3.csv`。
4. 為每一題畫出對應的樹形圖，輸出為 `tree_1.png`、`tree_2.png`... 等檔案。
5. 在終端機印出每題的 preorder / inorder / postorder 序列與圖片檔名。

---

## 2.2 輸入檔 `input3.csv` 格式說明

實際使用的是「純文字 + 題號」格式，而不是標準 CSV 標頭，內容如下：

```
#1
A,B,C,D,E,F
C,B,A,E,D,F

#2
1,2,3,4,5
5,4,3,2,1

#3
R,L,LL,LR,RL,RR
LL,L,LR,R,RL,RR

#4
a,b,c,d,e,f,g,h,i
c,b,e,d,a,g,f,i,h

#5
root
root
```

規則說明：

- 每一題以 `#題號` 開頭，例如 `#1`、`#2`。
- 接下來兩行為此題的
  - **preorder**：節點依前序走訪順序，以逗號分隔。
  - **inorder**：節點依中序走訪順序，以逗號分隔。
- 節點標籤可為任意字串（字母或數字），程式當作字串處理。
- 題與題之間可以有空白行，程式會自動略過。

---

## 2.3 輸出檔 `output3.csv` 格式說明

`output3.csv` 由程式產生，內容格式為：

```
question,postorder
1,C B E F D A
2,5 4 3 2 1
3,LL LR L RR RL R
```

```
4,c e d b g i h f a
5,root
```

- **question**: 題號 ("1" ~ "5") 。
- **postorder**: 以空白分隔的後序走訪節點序列。

---

## 2.4 輸出的樹形圖 `tree_*.png`

每題會輸出一張樹形圖，檔名為：

- 題目 1 → `tree_1.png`
- 題目 2 → `tree_2.png`
- 題目 3 → `tree_3.png`
- 題目 4 → `tree_4.png`
- 題目 5 → `tree_5.png`

範例圖示（預覽）：

### 題目 1



### 題目 2



### 題目 3



### 題目 4



### 題目 5



---

## 三、其他編譯語言之編譯方式 (Q3)

(Compile method if you use any other compiled language)

本題完全以 **Python** 實作，屬於直譯式語言，**不需要額外編譯步驟**。

- 只要有 `Q3.py` 與 `input3.csv` 的資料夾中執行：

```
python Q3.py
```

若未來在同一個 repo 中加入以 C / C++ / Java 實作的版本，請在此節補上對應的編譯與執行指令。

## 四、類別與函式說明 (Q3)

(Classes & functions description)

### 4.1 類別 `Node`

```
class Node:
 def __init__(self, val: str):
 self.val: str = val
 self.left: Optional["Node"] = None
 self.right: Optional["Node"] = None
```

- **用途：**表示二元樹的節點。
- **欄位：**
  - `val`: 節點標籤 (字串)。
  - `left`: 左子節點 (或 `None`)。
  - `right`: 右子節點 (或 `None`)。

### 4.2 `build_tree(preorder: List[str], inorder: List[str]) -> Optional[Node]`

- **功能：**

給定前序與中序走訪序列 (節點值不重複)，重建對應的二元樹並回傳根節點 `root`。
- **作法：**
  1. 建立 `in_index` 字典，對應「節點值 → 在 `inorder` 中的索引」。
  2. 使用遞迴 `helper(in_left, in_right)`:
    - 每次從 `preorder[pre_idx]` 取出新的根節點值。
    - 在 `inorder` 中找到其索引 `idx`，左邊為左子樹，右邊為右子樹。
- **回傳值：**
  - 二元樹根節點 `Node`，若輸入序列空則回傳 `None`。

### 4.3 `postorder_traversal(root: Optional[Node], res: List[str])`

- **功能：**

遞迴地對整棵樹進行 **後序走訪** (左 → 右 → 根)，將節點值依序加入 `res`。

### 4.4 `collect_edges(root: Optional[Node]) -> List[Tuple[str, str]]`

- **功能：**

掃描整棵樹，回傳所有邊 (`parent`, `child`) 的列表，用於畫圖。

- **方法:**
    - 以 DFS 遞迴，若節點有 `left` 或 `right`，就把 `(node.val, child.val)` 加入 `edges`。
- 

4.5 `compute_positions(root: Optional[Node]) -> Dict[str, Tuple[float, float]]`

- **功能:**  
為每個節點計算一個 `(x, y)` 座標，供 `networkx` 繪圖使用。
  - **策略:**
    - 以 `inorder` 遞迴：走訪左子樹 → 指定當前 `x` → 走訪右子樹。
    - `x` 值：全域計數器 `x_counter`，每拜訪到一個節點遞增 1。
    - `y` 值：使用 `depth`（根節點 `depth=0`，越往下越負）。
- 

4.6 `draw_tree(root: Optional[Node], filename: str) -> None`

- **功能:**  
使用 `networkx` + `matplotlib` 繪製樹，並儲存為 PNG 檔 `filename`。
  - **流程:**
    1. 呼叫 `collect_edges()` 取得邊。
    2. 使用 `nx.DiGraph()` 建圖並加入所有節點與邊。
    3. 呼叫 `compute_positions()` 取得節點位置。
    4. `nx.draw(G, pos, with_labels=True, arrows=False, node_size=800)` 繪圖。
    5. 使用 `plt.savefig(filename)` 儲存，`plt.close()` 關閉圖像。
- 

4.7 `read_input3(filename: str = "input3.csv")`

- **功能:**  
解析 `input3.csv`，回傳每題的 `(preorder, inorder)`。
  - **回傳型別:**
    - `Dict[str, Tuple[List[str], List[str]]]`  
例如：`data["1"] = (["A", "B", "C", ...], ["C", "B", "A", ...])`。
- 

4.8 `write_output3(posts: Dict[str, List[str]], filename: str = "output3.csv")`

- **功能:**  
將各題的 `postorder` 寫入 `output3.csv`。
  - **格式:**
    - 第一行標題：`question,postorder`
    - 之後每行：題號,以空白分隔的 `postorder` 串。
- 

4.9 `main()` 與 `if __name__ == "__main__":`

- **`main()` 功能:**

1. 呼叫 `read_input3("input3.csv")` 讀入所有題目。
2. 對每題：
  - 用 `build_tree()` 建樹。
  - 用 `postorder_traversal()` 得到 postorder 串。
  - 存入 `postorders[q]`。
  - 呼叫 `draw_tree()` 生成 `tree_q.png`。
  - 在終端機印出三種走訪序列與圖片檔名。
3. 呼叫 `write_output3(postorders, "output3.csv")` 輸出結果。

- 入口點：

```
if __name__ == "__main__":
 main()
```

使用時只要在專案根目錄執行：

```
python Q3.py
```

即可完成 Q3 要求的所有動作。